

Sift — Unified Roadmap

The single roadmap from the current **v1 proof-of-concept** to a complete, production-grade **AI chief of staff for creators**. It merges two axes of the same gap:

- **Axis A — Production-grade:** make what already exists real and reliable (hosting, billing, auth, ops, legal). Plumbing.
- **Axis B — Agent features:** make the product do more — from a search tool into an agent that advises, drafts, replies, and closes the loop. Capability.

v1 ships unchanged. The only thing either axis asks of v1 today is the cheap groundwork in **Part 1**. Everything else is additive.

Positioning: from "Search Console for your creator business" (passive) → **"Your AI chief of staff — Sift listens to your audience, tells you what they want, and helps you build it."**

Status legend: **built** built · **partial** partial (foundation exists) · — not started Priority (Axis A): P0 before launch · P1 to scale · P2 growth Release (Axis B): v1.5 · v2 · v3

The agent loop

v1 has three audience-facing steps; the agent adds three that turn insight into action:

1 LISTEN	search + comments	← v1 built
2 SIFT	intent, filter, trends	← v1 built (+ Part 1 foundations)
3 SERVE	grounded answer to fan	← v1 built
4 ADVISE	actionable insight	← agent (v1.5)
5 EXECUTE	draft + links	← agent (v1.5–v2)
6 CLOSE	notify asker + measure	← agent (v2)

The loop closes when the original asker hears back — which seeds the next question.

Part 1 — Foundations (the ONLY thing that touches v1)

Build these into the v1 schema now. They cost almost nothing and are the difference between the agent layer being a feature set and being a rebuild.

Foundation	What it does	Status today
Demand clustering & dedup	Merge similar questions into one cluster ("142 people asked this").	built workers/radar.py
Intent / outcome event log	Log who asked what, what was served, what the creator did, what happened next — the outcome edge.	partial events table logs isolated events; not yet linked into chains
Demand-item state machine	Each "useful" Radar card moves Idea → Drafting → Published → Loop-closed.	partial DemandTopic.creator_marked only; no lifecycle
Voice memory	Store approved replies/hooks and the creator's edits to drafts.	— not started

Concrete v1 work (small, additive): add a `status` lifecycle to `DemandTopic`; extend the event log into linked outcome records; add a voice-memory table. This is the recommended immediate next build.

Part 2 — Production-grade (Axis A)

Turn the v1 POC into a real, multi-tenant, self-serve product. (Companion plain-language version: `GOING_LIVE.md`.)

2.1 Onboarding & Authentication

Item	Pri	Status	Notes
Instagram OAuth sign-in flow (UI)	P0	—	Tokens are pasted via CLI today; token-exchange helpers exist in <code>instagram.py</code> .
Long-lived token refresh job	P0	—	IG tokens expire (~60 days); refresh before expiry or creators silently disconnect.
Token encryption at rest	P0	partial	Stored plaintext; encrypt with a KMS/secret key.
Creator login / session management	P0	—	Real accounts, not a single shared admin token.
YouTube OAuth flow	P1	—	Same pattern for owned channels.
Self-serve signup + onboarding wizard	P1	—	Connect → backfill progress → label questions → publish link.
Role-based access (creator/operator/admin)	P1	—	Today: one global bearer token.

2.2 Multi-Tenancy, Billing & Plans

Item	Pri	Status	Notes
Subscription billing (Stripe)	P0	—	Creator (~\$49-99) & Pro (~\$149-299) tiers; checkout, webhooks, dunning.
Plan-based feature gating	P1	—	e.g. comment-to-DM + storefront are Pro-only.
Usage metering & quotas	P1	—	Cap AI spend per creator; meter searches/DMs.
Free-trial / paid-pilot logic	P1	—	Convert pilots once Radar shows value.
Per-creator custom domains	P2	—	Path-based <code>/<handle></code> ships first.

2.3 Infrastructure & Operations

Item	Pri	Status	Notes
Production deployment (cloud)	P0	partial	docker-compose exists; need managed Postgres, object storage, TLS, domain.
Database migrations (Alembic)	P0	partial	Today schema is create-from-metadata; adopt versioned migrations before live data.
Secrets management	P0	—	Move keys/tokens out of <code>.env</code> .
CI/CD pipeline	P0	—	Tests + eval gate + lint on every push; automated deploys.
Monitoring, logging, alerting	P0	—	Error tracking, uptime, AI-cost dashboards, queue-depth alerts.
Backups & disaster recovery	P0	—	Automated DB backups + restore drills.
Background-job reliability	P1	partial	RQ + scheduler exist; add retries, dead-letter, durable scheduler.
Event analytics pipeline	P1	partial	<code>events</code> table exists; wire PostHog/warehouse.
Rate limiting & abuse protection	P1	—	Throttle public search; bot protection.
Caching / CDN for the fan page	P1	—	Cache result pages; CDN the Qwik app.

2.4 Legal, Privacy & Compliance

Item	Pri	Status	Notes
Privacy policy + ToS	P0	—	Required by Meta App Review and GDPR.
GDPR/CCPA deletion & export	P0	partial	Audience data pseudonymized + post-deletion purges chunks; need a user-facing request path.
Meta App Review submission	P0	—	Advanced Access for external creators; request only scopes the demo uses.
Data processing agreements	P1	—	DPA's with AI/infra subprocessors.
Content moderation / PII filters	P1	partial	No-answer + citation gate reduce hallucination; add abuse/PII filters on input.

2.5 Creator Dashboard & Quality

Item	Pri	Status	Notes
Demand Radar review UI	P0	—	Cards + useful/not/made buttons. API exists; no screen.
DM approval inbox (UI)	P0	—	Approve/edit/reject queued replies. API exists; no screen.
Wire real LLM + embedding + Whisper/OCR providers	P0	partial	Interfaces + adapters exist; need keys, prod config, GPU capacity.
Confidence-threshold calibration	P0	partial	Hand-set; calibrate against the labelled set per niche.
Larger eval sets wired into CI	P0	partial	Harness exists; gate regressions in CI.
Reranking model	P1	—	Today RRF fusion only; add a cross-encoder/LLM reranker.
Content/corpus browser, product manager, metrics charts	P1	partial	Metrics endpoint exists; needs screens.
Frontend + load tests	P1	—	Component/E2E tests; validate viral-post throughput.

Part 3 — Agent Feature Layer (Axis B)

The capability inventory from the Sift agent proposal. Each "Builds on" column points at the existing module it extends.

3.1 Audience & Demand Intelligence — "What does my audience want?"

Feature	Release	Builds on
The Briefing — pushed digest with ready hooks, re-promotion pick, gaps, trend alerts, one drafted post	v1.5	partial workers/ digest.py
Content gap map — answered well / partial / contradicted / never	v1.5	partial radar coverage check
Persona segmentation — cluster audience by behaviour	v2	partial clustering infra
Sentiment & emotion — anxiety / confusion / excitement	v2	—
Trend detection — rising themes; spike alerts only above a volume threshold	v2	partial WoW deltas
Cross-platform demand synthesis — newsletter/podcast/DM intake	v2	partial YouTube ingestion
Competitor demand radar — opt-in, aggregate, topic-level	v3	—

3.2 Content Production — "What should I make next?"

Feature	Release	Builds on
Content briefs — one-pager from a demand cluster	v1.5	partial radar + retrieval
Draft generator (script + hooks) — grounded in the creator's content + audience's words	v1.5	partial LLM + retrieval
Creator recall search — "where did I say that?"	v1.5	built retrieval (new surface)
Multi-format repurposing + Sift & Shift	v2	partial transcripts + LLM
Series builder	v2	partial radar
Performance prediction	v3	— needs history
Content calendar / planner	v3	—
Thumbnail & visual intelligence	v3	—

3.3 Engagement & Inbox — "What should I reply to, right now?"

Feature	Release	Builds on
Intent-labelled queue	v1.5	partial intent.py + dm_jobs
Bounded clarifying question	v1.5	partial retrieval confidence
No-answer waitlist	v1.5	partial no-answer state
Saved playbooks	v1.5	—
Reply assistant	v2	partial DM drafting
Agentic DM (approval mode)	v2	partial dm.py window/cap
Community delegation	v2	—
"Not now, but..." queue	v2	partial state machine (Part 1)
Loop-Closer — notify the original asker when published	v2	partial outcome log + state machine

3.4 Monetization & Revenue

Feature	Release	Builds on
Auto-affiliate injection — link + UTM at caption time	v1.5	built product→post mapping + tracked redirects
Offer-aware CTAs	v1.5	partial offer-awareness
Affiliate optimisation — A/B placement/anchor/pairings	v2	partial event tracking
Sponsor matchmaker & pitch	v2	partial radar aggregates
Dynamic pricing & offer insights	v3	—
Revenue forecasting & goals	v3	—
Brand partnership portal	v3 · scale	—

3.5 Brand, Growth, Operations

Feature	Release	Builds on
Crisis / sentiment-shift detection	v2	—
Voice consistency scoring	v2	partial voice memory (Part 1)
Plagiarism / unauthorised-use monitoring	v3	—
Content strategy advisor	v2	partial accumulated data
Peer benchmarking (opt-in/anonymised)	v3	—
Education & skill planning	v3	—
Task / project integration (Notion/Asana/Linear)	v3	partial state machine
Customer CRM	v3	—
Team & role-based access	v3	partial ties to §2.1 auth
Creator marketplace	v3 · scale	—
Public API & dev ecosystem	v3 · scale	partial FastAPI foundation

Gated on scale, not the calendar: brand portal, marketplace, and public API only matter at multi-creator scale — pursue only once the single-creator loop is proven and repeatable.

Part 4 — What to deliberately AVOID

Excluded	Why
Open-ended chat	Hallucination/brand risk; the bounded answer card captures most of the value safely.
Auto-posting without approval	Too much trust too early. Approval mode stays default through v3.
Real-time trend alerts (single creator)	Low signal-to-noise. Revisit only above a query-volume threshold.
Native video editing	Scope creep. Production stays at scripts/hooks/clip references.
Cross-platform identity resolution	Privacy-radioactive under GDPR. Topic-level demand synthesis only, never individual matching.

Part 5 — Guardrails that carry over (already enforced in code)

- **Retrieval correctness is the gate** — citation correctness must pass on a held-out set before any automation (`eval/ harness, --gate`).
- **Voice is a post-retrieval rewrite** — rewrite a correct, cited answer into the creator's tone; never generate novel claims. Correctness first, style second.
- **Approval mode by default** — `dm_jobs` start `pending_approval` ; bulk approval comes later as trust is earned.
- **DM limits shape Close** — one creator-initiated message per comment within 7 days; re-contact only via opted-in channels or inside that window (`dm.py`).
- **Privacy stays aggregate and opt-in** — pseudonymized + clustered (`privacy.py`); cross-platform/competitor features opt-in; deletion path + policy before public launch.

Part 6 — Unified sequencing

Both axes on one timeline. Production work (Axis A) and feature work (Axis B) interleave: productionize enough to run one creator, then layer capability.

Phase	Axis A — production	Axis B — features
v1 • now	(Part 1 groundwork: state machine, outcome log, voice-memory capture)	—
Phase A • make it real (P0)	Real AI keys → cloud deploy + managed Postgres → Alembic migrations → secrets → monitoring/backups → privacy policy → Radar review UI + DM approval inbox	—
v1.5 • assist	OAuth sign-in + token refresh → creator login → email digest delivery → Stripe billing → Meta App Review	The Briefing, content gap map, content briefs, draft + hook generator, creator recall search, bounded clarifying, no-answer waitlist, intent-labelled inbox, saved playbooks, auto-affiliate injection, offer-aware CTAs
v2 • agent	Plan gating & quotas → CI/CD → analytics → rate limiting/CDN	Reply assistant, agentic DM, community delegation, Loop-Closer, "not now" queue, persona segmentation, sentiment, trend detection, cross-platform synthesis, multi-format repurposing + Sift & Shift, series builder, sponsor matchmaker, affiliate optimisation, strategy advisor, crisis detection, voice scoring
v3 • scale	Team/roles, custom domains, DPAs	Performance prediction, content calendar, thumbnail intelligence, dynamic pricing, revenue forecasting, customer CRM, task/PM integration, peer benchmarking, education planning, competitor radar, plagiarism monitoring, brand portal, marketplace, public API

Part 7 — When to introduce Rust (backend language strategy)

The v1 spec prescribed **Python (FastAPI) + RQ/Celery primary, with Rust (Axum) for hot paths "when latency and cost demand it."** The build is Python today — correctly, because at POC scale nothing yet demands Rust. This section records the deliberate decision and the trigger to revisit, so it isn't relitigated.

Why Python now: the app is **I/O-bound, not CPU-bound**. A search-with-answer request is dominated by the LLM call (~0.5–2 s) plus network embedding and Postgres queries; the web-framework overhead is ~1 ms. Rewriting in Rust would change end-to-end latency by **well under 1%** while costing a ~5k-line rewrite and slowing iteration during the prove-the-bet phase. The heavy CPU work (Whisper, OCR, embeddings, LLM SDKs) also lives in the Python ML ecosystem, so the AI pipeline stays Python regardless — a "Rust backend" really means a **Python + Rust hybrid**, exactly as the spec described.

Where Rust genuinely helps — later, at multi-creator scale: - The **webhook ingestion path** under burst load (a viral post flooding comments) — higher throughput, lower tail latency. - The **search-serving endpoint** at high concurrency — less memory/CPU per request → fewer, cheaper servers. - **Cost at scale**, not speed-per-request — the real Rust win.

The trigger (measurement, not a date): introduce Rust for a specific path only when monitoring shows that path under real latency/cost pressure — e.g. webhook p99 latency breaching target under burst, or search-serving CPU/memory driving server count and cost. The architecture is already service-split (`apps/api` , `apps/workers`) behind one Postgres, so a single endpoint (the webhook receiver or the search route) can be ported to Axum **without touching the AI pipeline**. Port the one hot path the data points to; leave the rest.

Until that signal appears, Rust is premature optimization. Revisit at the v2 "agent" phase when traffic and cost data exist, or sooner if load testing (§2.3) surfaces a bottleneck.

How the docs fit together

- **IMPLEMENTATION_PLAN.md** — the v1 technical plan (what got built).
- **ROADMAP.md** (this file) — the unified path from v1 to the full agent product.
- **GOING_LIVE.md** — the plain-language launch checklist for the first real creator (a focused view of Phase A).
- **README.md** — how to run it locally.

Release tags and priorities are a sequencing hypothesis, not commitments — each feature should pass the v1 validation gates before it ships.